

HelixSpecifier

HelixSpecifier

Spezifikationsgetriebene Entwicklung, die ihren eigenen Formalismus an den Arbeitsaufwand anpasst.

Zusammenfassung

HelixSpecifier ist eine Go-Engine, die drei Entwicklungsmethoden – SpecKits spezifikationsgetriebenen Workflow, Superpowers TDD-Disziplin und GSDs Meilenstein-Lebenszyklus – zu einem adaptiven Prozess vereint. Sie klassifiziert jede Aufgabe nach Aufwand und skaliert den Prozessumfang entsprechend nach oben oder unten.

Kurzbeschreibung

HelixSpecifier ist eine Engine für spezifikationsgetriebene Entwicklung, die für AI-Agenten konzipiert ist. Sie kombiniert SpecKit, Superpowers und GSD, klassifiziert Aufgaben nach Aufwand, führt debattengestützte Spezifikationsphasen durch, erzwingt ein Mindestverhältnis von Tests zu Code und lernt aus jedem abgeschlossenen Prozess.

Ausführliche Beschreibung

HelixSpecifier ist eine Engine für spezifikationsgetriebene Entwicklung (SDD), geschrieben in Go (Modul `digital.vasic.helixspecifier`) und als Komponente des HelixAgent-AI-Ensembles konzipiert. Sie vereint drei Entwicklungsansätze, die normalerweise in drei separaten Tools – und drei unterschiedlichen Denkweisen – verankert sind, zu einem adaptiven Workflow: SpecKits siebenphasigen SDD-Prozess (Constitution, Spezifizieren, Klären, Planen, Aufgaben, Analysieren, Implementieren), Superpowers testgetriebene Disziplin mit paralleler Subagenten-Ausführung sowie GSDs Meilenstein-Lebenszyklus-Management. Jeder dieser Pfeiler

erfüllt weiterhin seine ursprüngliche Funktion; die Engine sorgt jedoch dafür, dass sie als kohärenter Gesamtprozess ablaufen – statt als manuell zusammengefügt Workflow.

Ihr zentrales Konzept ist der *adaptive Formalismus*: Die Engine klassifiziert eingehende Aufgaben nach Aufwand und passt den Prozessumfang entsprechend an. So durchläuft eine Einzeilen-Korrektur nicht denselben aufwendigen Ablauf wie ein großes Feature – und ein großes Feature wird nicht mit der Sorglosigkeit einer Tippfehler-Korrektur abgehandelt. Zehn Kernfunktionen bauen auf diesem Grundprinzip auf: parallele Aufgabenausführung mit begrenzter Nebenläufigkeit, ein maschinenlesbares „Constitution as Code“, das verbindliche Regeln automatisch durchsetzt, „Nyquist-TDD“, das ein Mindestverhältnis von Tests zu Implementierung überwacht und erzwingt, mehrstufige Multi-Agenten-Debatten zur Spezifikationsverfeinerung, adaptive Kompetenzlernfähigkeit, Brownfield-Analyse von Legacy-Code, prädiktive Spezifikationen auf Basis historischer Muster, projektübergreifender Wissensaustausch, dynamische Anpassung des Formalismus während der Laufzeit sowie ein persistenter Spezifikationsspeicher mit semantischer Suche.

Die Engine wird als Go-Modul genutzt – entweder über `go get` oder eine lokale `replace`-Direktive – und ist hinter einer bewusst schlanken Engine-API verborgen: Man registriert die drei Pfeiler sowie einen Formalismus-Skalierer und einen Spezifikationsspeicher, klassifiziert den Aufwand der Aufgabe und führt den gesamten Prozess aus, um ein qualitätsbewertetes Ergebnis zu erhalten. Die Oberfläche ist einfach; die Orchestrierung dahinter ist es nicht. Wie der Rest der Helix-Familie wird sie unter einem Anti-Bluff-Verifizierungsregime entwickelt, bei dem ein In-Prozess-Challenge-Runner echten Code statt Mocks ausführt.

Warum wir sie entwickelt haben

Spezifikationsgetriebene Entwicklung, rigoroses TDD und Meilenstein-Management sind normalerweise drei separate Praktiken mit drei separaten Tools. HelixSpecifier wurde entwickelt, damit ein AI-Agent (HelixAgent) alle drei als einen kohärenten, selbstskalierenden Workflow nutzen kann – statt sie manuell zusammenfügen zu müssen.

Warum es ein Game-Changer ist

Es macht den Prozess automatisch proportional zur Arbeit. Teams stecken normalerweise in einer von zwei ungünstigen Extremsituationen fest: übermäßige Formalitäten bei allem

(sicher, aber langsam und heimlich verhasst) oder Formalitäten bei nichts (schnell, bis es das nicht mehr ist). HelixSpecifier löst diesen Kompromiss auf, indem es den Formalismus an den klassifizierten Aufwand jeder Aufgabe anpasst und ihn zur Laufzeit neu justiert, sobald sich die Arbeit offenbart. Die Fähigkeit, die vorher nicht praktikabel war, ist ein Prozess, der sich pro Aufgabe selbst optimiert – und darüber hinaus Spezifikationsentscheidungen, die nicht auf einer ersten Einschätzung eines Einzelnen beruhen, sondern auf einer mehrstufigen, multiagenten Debatte mit Positionsbewertung.

Was innovativ ist

- **Adaptiver Formalismus** – Prozessniveau wird durch Echtzeit-Qualitätsmetriken gesteuert und zur Laufzeit angepasst, nicht im Voraus festgelegt.
- **Nyquist-TDD** – ein Test-zu-Implementierungs-Verhältnis-Gate (mindestens 2:1), das sich die Logik des Nyquist-Shannon-Abtasttheorems zunutze macht: Um Verhalten treu abzubilden, muss man es deutlich häufiger abtasten als es auftritt – Tests müssen also das gemessene Codevolumen übertreffen.
- **Debattenarchitektur** – mehrstufige, multiagentenbasierte Spezifikationsverfeinerung, bei der Positionen vorgeschlagen, bewertet und konvergiert werden, um Einzelmeinungen durch einen adversariellen Prozess zu ersetzen.
- **Prädiktive Spezifikation und projektübergreifender Wissenstransfer** – die Engine analysiert akkumulierte Workflows, um Spezifikationen vorherzusagen und hart erarbeitetes Wissen von einem Projekt ins nächste zu übertragen.
- **Constitution als Code** – verbindliche Projektregeln werden maschinenlesbar gemacht und von der Engine durchgesetzt, statt auf die Wachsamkeit von Reviewern angewiesen zu sein.

Größte technische Herausforderungen & wie wir sie gelöst haben

- **Drei Methodologien zusammenführen, ohne dass sie sich gegenseitig behindern** – SpecKit, Superpowers und GSD gehen jeweils davon aus, den Workflow zu dominieren. Gelöst wurde dies mit einer Fusions-Engine, die jede Säule hinter einer gemeinsamen Schnittstelle registriert und sie durch einen gemeinsamen Prozesslebenszyklus steuert, sodass sie sich zu einem einzigen Ablauf verbinden, statt drei kollidierende Systeme zu bilden.
- **Entscheiden, wie viel Prozess eine bestimmte Aufgabe tatsächlich benötigt** – zu viel, und alles kriecht; zu wenig, und riskante Arbeit wird ungeprüft ausgeliefert. Gelöst mit

einem Aufwandsklassifikator, der die Arbeit bewertet und einen Formalismus-Skalierer speist, der das Prozessniveau dynamisch anpasst, während die Ausführung voranschreitet.

- **Spezifikationsqualität hochhalten, ohne bei jeder Entscheidung einen menschlichen Gatekeeper einzusetzen** – gelöst durch den Ersatz von Einmal-Spezifikationen durch debattengestützte Verfeinerung, bei der Agenten konkurrierende Positionen über mehrere Runden hinweg bewerten, sowie durch die Durchsetzung von Nyquist-TDD-Verhältnissen, damit die Implementierung nicht schneller voranschreitet als ihre Tests.

Technologie-Stack

- **Go** – gewählt, damit die Engine als einzelnes importierbares Binary ohne Laufzeit-Overhead ausgeliefert werden kann; sein Nebenläufigkeitsmodell macht die parallelisierte Aufgabenverteilung mit begrenzter Parallelität und die mehrstufigen Debattenrunden handhabbar, statt zu einem Threading-Albtraum zu werden.
- **logrus** – strukturierte Protokollierung, die durch die Engine und alle drei Säulen läuft, sodass Entscheidungen eines Workflows (Klassifizierung, Formalismus-Anpassungen, Debattergebnisse) im Nachhinein nachvollziehbar sind.
- **SpecKit-Säule** – der siebenphasige, spezifikationsgetriebene Entwicklungsprozess (Constitution → Spezifizieren → Klären → Planen → Aufgaben → Analysieren → Implementieren), der das disziplinierte Rückgrat dafür bildet, wie aus einer Spezifikation Code wird.
- **Superpowers-Säule** – TDD-Disziplin mit paralleler Subagenten-Ausführung, die für testgetriebene Strenge sorgt und durch Aufteilung die Implementierung ehrlich und schnell hält.
- **GSD-Säule** – Meilenstein- und Lebenszyklusmanagement, das dem Workflow sein Gefühl von „Fertigstellung“ und seinen Fortschritt durch die Phasen verleiht.
- **Spec-Memory-Store** – ein persistenter, semantisch durchsuchbarer Index vergangener Spezifikationen, das Substrat, das prädiktive Spezifikation und projektübergreifenden Wissenstransfer ermöglicht, statt jedes Mal bei null anzufangen.

Status- & Ehrlichkeitshinweise

- **Status: Beta.** Als Go-Modulkomponente von HelixAgent verwendet.
- **Lizenz: Noch nicht festgelegt.** Keine LIZENZ über den GitHub-API-Prozess erkannt — UNBESTÄTIGT / nicht deklariert.

- Der Anzeigename „*HelixSpecifier*“ verweist auf das Repository specifier.

Prioritätsstufe: Helix-primär.